Hyperparameter Tuning — Complete Guide

What is Hyperparameter Tuning?

Hyperparameter Tuning is the process of finding the **best configuration settings** for a machine learning model to maximize its performance!

Think of it like this:

You are baking a cake
You can adjust:
Temperature, Time, Amount of sugar, Amount of flour
These are your HYPERPARAMETERS!
Finding the PERFECT combination = Hyperparameter Tuning!

Super Simple Real Life Analogies

Radio Tuning Analogy

Old radio has a tuning dial
You rotate it to find the CLEAREST signal
Too left → static noise
Too right → different static noise
Just right → PERFECT CLEAR SIGNAL!

Hyperparameter tuning = finding that perfect dial position!

Gym Training Analogy

You want to get fit
Hyperparameters to tune:
→ How many reps (iterations)?
→ How heavy the weights (learning rate)?
→ How many sets (layers)?
→ Rest time between sets (regularization)?

Wrong combination → poor results or injury!
Right combination → maximum fitness!

Chemistry Lab Analogy

Scientists mix chemicals
Must find perfect:
→ Temperature

→ Pressure
→ Time
→ Quantities

Wrong mix → failed experiment!
Right mix → perfect reaction!

Parameters vs Hyperparameters

Parameters (Learned by Model)

Parameters are learned AUTOMATICALLY

during the training process!

Examples:

→ Weights in Neural Networks (w_1, w_2, ...)

→ Coefficients in Linear Regression (β_0, β_1, ...)

→ Support vectors in SVM

→ Split values in Decision Trees

You do NOT set these!

Model LEARNS them from data!

They change during training!

Hyperparameters (Set by YOU Before Training)

Hyperparameters are set by YOU

BEFORE training begins!

They CONTROL how the model learns!

Model does NOT learn them from data!

YOU must choose them!

Sumit Sir Notes

Examples:

→ K in KNN (how many neighbors?)

→ Max depth in Decision Tree (how deep?)

→ Learning rate in Neural Networks (how fast to learn?)

→ C and gamma in SVM (how strict?)

→ n_estimators in Random Forest (how many trees?)

→ Alpha in Ridge/Lasso (how much regularization?)

Setting wrong hyperparameters:

→ Overfitting (too complex)

→ Underfitting (too simple)

→ Slow or no convergence

→ Poor generalization!

Parameters vs Hyperparameters Table

Feature	**Parameters**	**Hyperparameters**
Who sets them?	Model (automatic)	You (manual)
When set?	During training	Before training
How learned?	Gradient descent, optimization	Search strategies
Examples	Weights, coefficients	Learning rate, K, depth
Change during training?	Yes	No

Why Hyperparameter Tuning Matters?

Same algorithm, DIFFERENT hyperparameters:

Random Forest with default settings:

Accuracy = 75%

Random Forest after tuning:

n_estimators = 200

max_depth = 10

min_samples_split = 5

max_features = 'sqrt'

Accuracy = 91%

16% improvement just from tuning!

No new data, no new algorithm!

Just better hyperparameters!

This is why tuning is SO important!

PART 1 — Common Hyperparameters by Algorithm

KNN Hyperparameters

1. n_neighbors (K):

→ Number of nearest neighbors

→ Too small: overfit

→ Too large: underfit

→ Typical range: 1 to 30

→ Best found by CV!

2. weights:

→ 'uniform': all neighbors equal

→ 'distance': closer = more weight

→ Usually 'distance' is better

3. metric:

→ 'euclidean': straight-line distance

→ 'manhattan': grid distance

→ 'minkowski': generalization

→ 'cosine': for text data

4. algorithm:

→ 'auto': sklearn decides

→ 'ball_tree': high dimensions

→ 'kd_tree': low dimensions

→ 'brute': small datasets

5. p:

→ Power for Minkowski metric

→ p=1 → Manhattan

→ p=2 → Euclidean (default)

Decision Tree Hyperparameters

1. max_depth:

→ Maximum depth of tree

→ None: grows until pure leaves (overfit!)

→ 3-10: typical range

→ Controls complexity!

2. min_samples_split:

→ Minimum samples to split a node

→ Higher → simpler tree (less overfit)

→ Typical range: 2 to 20

3. min_samples_leaf:

→ Minimum samples at leaf node

→ Higher → smoother predictions

→ Typical range: 1 to 20

4. max_features:

→ Features to consider at each split

→ 'sqrt': √(n_features)

→ 'log2': $\log_2$(n_features)

→ int/float: specific number

→ Controls randomness!

5. criterion:

→ 'gini': Gini impurity

→ 'entropy': Information gain

→ 'log_loss': Log loss

6. max_leaf_nodes:

→ Maximum number of leaf nodes

→ Controls tree size directly!

Random Forest Hyperparameters

1. n_estimators:

→ Number of trees in forest

→ More trees = better (diminishing returns!)

→ Typical range: 100 to 1000

→ Start with 100!

2. max_depth:

→ Depth of each tree

→ None: fully grown

→ 5-20: typical range

3. min_samples_split:

→ Minimum samples to split

→ Higher → simpler trees

→ Range: 2 to 20

4. min_samples_leaf:

→ Minimum samples at leaf

→ Higher → smoother model

→ Range: 1 to 20

5. max_features:

→ Features per split

→ 'sqrt' (classification default)

→ 'log2'

→ Float (fraction)

6. bootstrap:

→ True: use bootstrap samples

→ False: use full dataset

→ Usually True!

7. class_weight:

→ None: equal weights

→ 'balanced': handles imbalance

→ Custom dict

8. max_samples:

→ Fraction of samples per tree

→ For very large datasets

SVM Hyperparameters

1. C (Regularization):

→ Controls margin vs misclassification

→ Large C: small margin, fits training (overfit!)

→ Small C: large margin, more misclass (underfit!)

→ Typical range: 0.001 to 1000

→ MOST IMPORTANT parameter!

2. kernel:

→ 'linear': linear boundary

→ 'rbf': Gaussian (most common!)

→ 'poly': polynomial boundary

→ 'sigmoid': neural-network-like

3. gamma (for rbf, poly, sigmoid):

→ Controls influence of single sample

→ Large gamma: close points influence

→ complex boundary (overfit!)

→ Small gamma: far points influence

→ smooth boundary (underfit!)

→ 'scale': 1/(n_features × X.var())

→ 'auto': 1/n_features

4. degree (for poly kernel):

→ Degree of polynomial

→ Higher → more complex

→ Typical: 2, 3

5. coef0 (for poly, sigmoid):

→ Independent term in kernel function

6. class_weight:

→ 'balanced' for imbalanced data

Neural Network Hyperparameters

Architecture:

1. Number of layers:

→ More layers: deeper, more complex

→ Typical: 2-5 hidden layers

2. Neurons per layer:

→ More neurons: more capacity

→ Powers of 2: 32, 64, 128, 256, 512

3. Activation functions:

→ ReLU (default hidden layers)

→ Sigmoid (binary output)

→ Softmax (multi-class output)

→ Tanh (sometimes better than sigmoid)

Training:

4. Learning rate:

→ How fast to update weights

→ Too high: overshoots, diverges!

→ Too low: very slow learning!

→ Typical: 0.001 to 0.1

→ MOST CRITICAL parameter!

5. Batch size:

→ Samples per gradient update

→ Small: noisy but generalizes better

→ Large: stable but may overfit

→ Common: 32, 64, 128, 256

6. Epochs:

→ Number of full dataset passes

→ Use early stopping instead!

7. Optimizer:

→ SGD: simple, needs tuning

→ Adam: adaptive, usually works best

→ RMSprop: good for RNNs

Regularization:

8. Dropout rate:

→ Fraction of neurons dropped

→ Typical: 0.2 to 0.5

9. L1/L2 regularization (weight decay):

→ Penalizes large weights

→ Typical: 1e-4 to 1e-2

10. Early stopping patience:

→ Epochs to wait before stopping

→ Typical: 5 to 20

Gradient Boosting (XGBoost/LightGBM) Hyperparameters

1. n_estimators:

→ Number of boosting rounds/trees

→ More trees with small learning rate!

→ Typical: 100 to 1000

2. learning_rate (eta):

→ Shrinks each tree's contribution

→ Lower: more trees needed, better!

→ Typical: 0.01 to 0.3

→ Inverse with n_estimators!

3. max_depth:

→ Depth of each tree

→ Typical: 3 to 10

→ XGBoost: deeper ok

→ LightGBM: uses leaves not depth

4. subsample:

→ Fraction of samples per tree

→ 0.5-1.0: controls variance

→ Like Random Forest's bootstrap!

5. colsample_bytree:

→ Fraction of features per tree

→ 0.5-1.0: like Random Forest max_features

6. min_child_weight:

→ Minimum instances in leaf

→ Higher: more conservative (less overfit)

7. gamma / min_split_loss:

→ Minimum loss reduction to split

→ Higher: more conservative

8. reg_alpha (L1 regularization):

→ Lasso-like regularization

→ Typical: 0 to 1

9. reg_lambda (L2 regularization):

→ Ridge-like regularization

→ Typical: 0 to 10

10. scale_pos_weight:

→ For imbalanced data

→ = n_negative / n_positive

PART 2 — Tuning Strategies

Overview of All Strategies

Hyperparameter Tuning Strategies:

```
|
├── Manual Search
|   → Human intuition and experience
|
├── Grid Search
|   → Try ALL combinations
|
├── Random Search
|   → Try RANDOM combinations
|
```

PART 3 — Manual Search

What is Manual Search?

Setting hyperparameters by hand using domain knowledge, intuition, and trial and error!

How Manual Search Works

Step 1: Start with default hyperparameters

→ sklearn defaults are usually reasonable!

Step 2: Understand each hyperparameter's effect

→ Know what each controls!

Step 3: Train and evaluate baseline model

Step 4: Change ONE hyperparameter at a time

→ See how it affects performance

Step 5: Use intuition and rules of thumb

→ e.g., if overfitting → reduce complexity

Step 6: Repeat until satisfied

Example for Random Forest:

Start: n_estimators=100, max_depth=None

→ Training: 99%, CV: 75% → Overfitting!

Add: max_depth=5

→ Training: 85%, CV: 83% → Better!

Add: min_samples_leaf=5

→ Training: 82%, CV: 84% → Even better!

Rules of Thumb for Manual Tuning

If model OVERFITS (train >> test):

→ Decrease max_depth

→ Increase min_samples_split/leaf

→ Increase regularization (C smaller for SVM)

→ Decrease n_estimators (careful!)

→ Increase dropout (Neural Networks)

→ Reduce number of features

→ Increase training data

If model UNDERFITS (train ≈ test, both low):

→ Increase max_depth

→ Decrease min_samples_split/leaf

→ Decrease regularization

→ Increase n_estimators

→ Add more features or feature engineering

→ Try more complex model

→ Decrease dropout

If model is SLOW TO CONVERGE:

→ Increase learning rate

→ Change optimizer

→ Normalize features

→ Use batch normalization

If training is UNSTABLE:

→ Decrease learning rate

→ Increase batch size

→ Add gradient clipping

Pros and Cons of Manual Search

Pros:

→ Fast to implement

→ Uses domain knowledge

→ Good starting point

→ Educational (understand model behavior)

→ No computation overhead

Cons:

→ Time-consuming for humans

→ Misses optimal combinations

→ Biased by human assumptions

→ Cannot search large parameter spaces

→ Reproducibility issues

→ Inconsistent results

PART 4 — Grid Search

What is Grid Search?

Grid Search tries **ALL possible combinations** of specified hyperparameter values — exhaustive search!

Grid Search Analogy

Map Grid Analogy

You lost your keys in a room
Grid search = check EVERY square meter
systematically, one by one!
GUARANTEED to find the best!
But takes LONG TIME!

How Grid Search Works

Define parameter grid:

```
param_grid = {
   'n_neighbors': [3, 5, 7, 9],
   'weights': ['uniform', 'distance'],
   'metric': ['euclidean', 'manhattan']
}
```

Total combinations:

4 × 2 × 2 = 16 combinations!

With 5-fold CV:

16 × 5 = 80 model trainings!

Grid of combinations:

	uniform		distance	
	eucl	manhat	eucl	manhat
K=3	✓	✓	✓	✓
K=5	✓	✓	✓	✓
K=7	✓	✓	✓	✓
K=9	✓	✓	✓	✓

All 16 boxes checked!

Best combination selected!

Grid Search Step-by-Step

Step 1: Define hyperparameter grid

Think carefully about ranges!

Too large grid → very slow!

Step 2: For EACH combination:

→ Run K-Fold Cross Validation

→ Get mean CV score

→ Store result

Step 3: Find combination with best score

Step 4: Retrain on full training data

with best hyperparameters

Step 5: Evaluate on test set ONCE

Example:

Combination 1: K=3, uniform, euclidean → 82%

Combination 2: K=3, uniform, manhattan → 81%

Combination 3: K=3, distance, euclidean → 83%

...

Combination 10: K=7, distance, euclidean → 88% ← BEST!

...

Combination 16: K=9, distance, manhattan → 84%

Best: K=7, distance, euclidean with 88% accuracy!

Computational Cost of Grid Search

Parameters:

p_1 has n_1 values

p_2 has n_2 values

p_3 has n_3 values

Total combinations = $n_1 \times n_2 \times n_3 \times ...$

With K-fold CV:

Total trainings = $n_1 \times n_2 \times n_3 \times K$

Example:

Random Forest:

n_estimators: [100, 200, 300, 500] → 4 values

max_depth: [3, 5, 7, 10, None] → 5 values

min_samples_split: [2, 5, 10] → 3 values

min_samples_leaf: [1, 2, 4] → 3 values

max_features: ['sqrt', 'log2', 0.5] → 3 values

Total = 4×5×3×3×3 = 540 combinations!

With 5-fold CV: 540 × 5 = 2700 trainings!

For complex models (XGBoost, Neural Networks):

Each training takes minutes!

2700 × 5 minutes = 9000 minutes = 6.25 days!

This is why Grid Search is impractical

for large parameter spaces!

Pros and Cons of Grid Search

Pros:

→ Guaranteed to find best combination

in the specified grid!

→ Systematic and exhaustive

→ Reproducible

→ Simple to understand and implement

→ Good for small parameter spaces!

Cons:

→ Exponential time complexity!

→ Cannot search continuous spaces well

→ Misses combinations NOT in grid

→ Very slow for large parameter spaces

→ Wasteful (tests many poor combinations)

→ Curse of dimensionality!

Best used when:

→ 2-3 hyperparameters to tune

→ Small number of values each

→ Fast model (logistic regression, small trees)

→ Computation is not a constraint

PART 5 — Random Search

What is Random Search?

Random Search samples **random combinations** of hyperparameters instead of trying all — more efficient than Grid Search!

Random Search Analogy

Random Sampling Analogy

Lost keys in large room (hotel ballroom!)
Grid search: check every inch
Random search: randomly check 50 different spots!

Key insight from research:
"Not all hyperparameters matter equally!
A few matter a lot, most matter little!"

Random search is much more likely to
find good values for IMPORTANT params!

How Random Search Works

Define parameter distributions:

```
param_distributions = {
   'n_neighbors': range(1, 31),       ← 30 values
   'weights': ['uniform', 'distance'],  ← 2 values
   'metric': ['euclidean', 'manhattan', 'cosine'] ← 3 values
}
```

Total possible: 30 × 2 × 3 = 180 combinations

Random Search with n_iter=20:

→ Randomly sample 20 combinations from 180!

→ Evaluate each with K-fold CV

→ Pick best one!

20 trainings instead of 180!

10x faster!

Randomly sampled combinations:

Combo 1: K=15, distance, euclidean → 85%

Combo 2: K=3, uniform, cosine → 79%

Combo 3: K=23, distance, manhattan → 82%

Combo 4: K=7, distance, euclidean → 88% ← Best!

...

Combo 20: K=11, uniform, euclidean → 86%

Best found: K=7, distance, euclidean → 88%!

Why Random Search is Better than Grid Search

Key Research Finding (Bergstra & Bengio, 2012):

"Random search outperforms grid search

in most practical scenarios!"

Why?

Most hyperparameters are NOT equally important!

Usually 1-2 hyperparameters dominate performance!

Grid Search:

Fixes less-important parameter values!

Wastes most trials on irrelevant combinations!

GRID SEARCH:

Param A (unimportant): ———┬———┬———┬———

Param B (important): ↕ ↕ ↕ ↕

b1 b2 b3 b4

Only 4 unique values for B tested!

RANDOM SEARCH:

Param A: anywhere ←← random ←← anywhere

Param B: anywhere ←← random ←← anywhere

20 DIFFERENT values for B explored!

Much better coverage of important param B!

Result: Random search finds nearly-optimal

solution in fraction of time!

Random Search with Distributions

For CONTINUOUS hyperparameters:

Use probability distributions!

from scipy.stats import uniform, loguniform

Learning rate: loguniform(0.001, 0.1)

→ Samples from log-uniform distribution

→ Equal probability to sample 0.001, 0.01, 0.1

→ Better than uniform for learning rate!

→ Most learning rates cluster near small values!

C for SVM: loguniform(0.01, 1000)

→ Better than uniform [0.01, 1000]

→ Uniform would almost always pick large C!

Dropout: uniform(0.1, 0.5)

→ Any value between 0.1 and 0.5

→ Fine-grained exploration!

Integer ranges:

n_estimators: randint(50, 500)

→ Any integer from 50 to 499

→ Much finer than grid [100, 200, 300]!

Choosing n_iter in Random Search

n_iter = number of random combinations to try

Budget-based approach:

→ Fixed time budget

→ How many trainings fit in budget?

Thumb rule:

n_iter = 2 × (number of hyperparameters) × 10

→ 3 hyperparameters → n_iter = 60

→ 5 hyperparameters → n_iter = 100

Research suggests:

60 iterations of Random Search finds

good hyperparameters 95% of the time!

Diminishing returns:

0-20 iter: Big improvements

20-60 iter: Moderate improvements

60-100 iter: Small improvements

100+ iter: Marginal improvements

More iterations = more compute = diminishing returns!

Grid Search vs Random Search

Feature	**Grid Search**	**Random Search**
Search type	Exhaustive	Random sampling
Finds global optimum?	In grid	Not guaranteed
Computation	Exponential	Linear (n_iter)
Continuous params	Poor	Excellent
Many params	Very slow	Handles well
Few params	Good	Overkill
Best for	Small grids	Large spaces
